

EPIC 05 — Inter-Service Communication & Resilience

JIRA Task 03 · Circuit Breaker & Fallback · Sprint 12 (21/9 – 4/10)

Objective

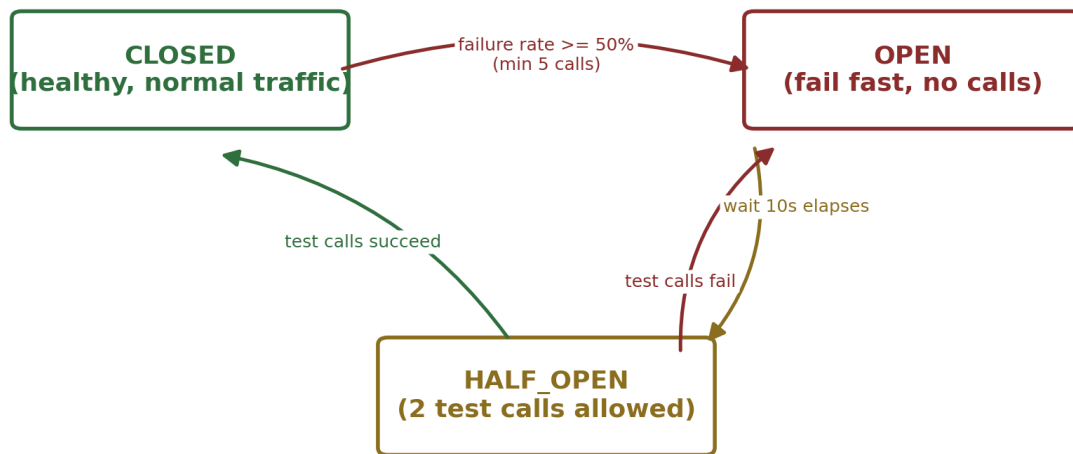
Understand and implement the Circuit Breaker pattern to prevent cascading failures between microservices. Where Task 02's retry logic handles brief, momentary blips, the Circuit Breaker handles the case where a dependency (User Service) is down for a longer stretch - instead of every request wasting time retrying against a service that clearly is not coming back soon, Order Service “gives up early” for a while and checks back periodically instead.

1. The Three States (Step 1)

State	Meaning
CLOSED	Normal operation. Requests flow through to User Service as usual - the default, healthy state.
OPEN	The breaker has tripped. Order Service stops even trying to call User Service for a while - it fails immediately, without wasting time waiting or retrying.
HALF_OPEN	After a cooldown period in OPEN, the breaker cautiously lets through a small number of test requests to check whether User Service has recovered, without fully committing to normal traffic again.

2. State Diagram (Step 2 & Step 10)

Circuit Breaker State Transitions



Configured for the Order Service -> User Service call
sliding window: 5 calls | failure threshold: 50% | wait in OPEN: 10s | test calls in HALF_OPEN: 2

3. Configuration (Steps 3 & 5)

Added the Resilience4j circuit breaker library (plus spring-boot-starter-aop, which it depends on) to Order Service's pom.xml. Thresholds configured in application.properties, using reasonable development values:

Setting	Value	Meaning
Sliding window	5 calls (count-based)	The breaker looks at the last 5 calls to judge recent health, not the entire history
Minimum calls	5	Do not judge health until at least 5 calls have actually happened - avoids overreacting to 1-2 early failures
Failure threshold	50%	If 50% or more of the last 5 calls failed, trip the breaker to OPEN
Wait duration (OPEN)	10 seconds	Once OPEN, stay closed-off for 10s before cautiously checking again
Permitted calls (HALF_OPEN)	2	When testing recovery, allow only 2 trial requests through before deciding whether to fully close or trip back open

4. Applying the Breaker (Step 4)

The `@CircuitBreaker` annotation wraps `UserServiceClient.getUserDetails()` - the same method that contains the Task 02 retry loop. This means the breaker treats “3 retries, all failed” as ONE failed call for its own counting purposes: retry absorbs brief blips, while the circuit breaker judges the bigger picture of whether User Service has been healthy lately.

```
@CircuitBreaker(name = "userService", fallbackMethod = "getUserDetailsFallback")
```

5. Fallback Design (Step 6)

Rather than failing the whole request when User Service is unreachable, the fallback lets the order request still succeed - returning the order data with user set to null and a clear message. This is graceful degradation: the API stays useful even when a dependency is down, instead of returning a hard error for something the client did not do wrong.

One exception: “user not found” (404) is a genuine, permanent error - not a service-availability problem - so it is rethrown as a real error instead of being silently treated as a fallback case.

```
{
  "orderId": 13,
  "productName": "Laptop",
  "quantity": 2,
  "amount": 50000.00,
  "status": "CREATED",
  "user": null,
  "message": "User service temporarily unavailable"
}
```

6. Test Evidence (Steps 7, 8, 9)

Step 7 - Healthy service (baseline)

With both services running, GET /orders/13/with-user returned 200 OK with the full order and nested user object, confirming the circuit breaker code does not interfere with normal, healthy traffic (circuit stays CLOSED).

Step 8 - Stop User Service, observe the breaker opening

User Service was stopped and the same request was sent repeatedly. The first several calls each ran the full Task 02 retry cycle before failing:

```
[UserServiceClient] Request started for userId=10
[UserServiceClient] Retry failure on attempt 1 (connection refused)
[UserServiceClient] Retry attempt 2 for userId=10
[UserServiceClient] Retry failure on attempt 2 (connection refused)
[UserServiceClient] Retry attempt 3 for userId=10
[UserServiceClient] Final failure after 3 attempts (connection refused)
[UserServiceClient] Circuit breaker fallback triggered for userId=10 (RuntimeException)
```

Once 5 calls had been counted and the 50% failure threshold was crossed, the breaker tripped to OPEN. Every subsequent call then skipped the retry loop entirely:

```
[UserServiceClient] Circuit breaker fallback triggered for userId=10
(CallNotPermittedException)
```

CallNotPermittedException is the proof the breaker was OPEN - Resilience4j blocked the call before it was even attempted, rather than trying and failing. Response time dropped from ~13-17 seconds (full retry cycle) to under 100ms once the breaker tripped - exactly the point of this pattern: stop wasting time on calls that are very likely to fail anyway.

Step 9 - Restart User Service, observe recovery

User Service was started again. After the 10-second wait-duration elapsed, the breaker automatically moved to HALF_OPEN and let the next two calls through as real test requests:

```
[UserServiceClient] Request started for userId=10
[UserServiceClient] Request succeeded on attempt 1
[UserServiceClient] Request started for userId=10
[UserServiceClient] Request succeeded on attempt 1
```

Both test calls succeeded (no retries needed, no CallNotPermittedException), so the breaker closed itself back to normal, healthy operation - fully self-healed with no manual intervention required.

Deliverables Checklist

- Circuit Breaker implementation - Resilience4j applied to the Order Service -> User Service call
- Fallback method - graceful degradation (user: null + message) instead of a hard failure
- Threshold configuration - sliding window, minimum calls, failure rate, wait duration, half-open test calls, all documented with values and reasoning
- Test scenarios - healthy service, breaker opening under sustained failure, and automatic recovery via half-open, all verified with terminal log evidence

- State diagram - CLOSED / OPEN / HALF_OPEN transitions documented with triggers for each
- Documentation - this document